

GYMNASIUM JANA KEPLERA

Parléřova 2/118, 169 00 Praha 6



Arbitra

Maturitní práce

Autor: Gabriel Hamrle

Třída: R8.A

Školní rok: 2025/2026

Předmět: Informatika

Vedoucí práce: Emil Miler

Praha, 2026



Gymnázium Jana Keplera

Kabinet informatiky

ZADÁNÍ MATURITNÍ PRÁCE

- *Student:* Gabriel Hamrle
 - *Třída:* R8.A
 - *Školní rok:* 2025/2026
 - *Vedoucí práce:* Emil Miler
 - *Název práce:* Arbitra
 - *URL repozitáře:* <https://github.com/Dxlaby/Arbitra>
-

Pokyny pro vypracování:

Cílem této práce je vytvořit program, který bude extrahovat data z webových stránek sázkových kanceláří, zejména kurzy na jednotlivé sportovní zápasy. Program bude schopen rozpoznat totožné události nabízené různými sázkovými kancelářemi, určit pro každou z nich nejvyšší dostupný kurz a případně identifikovat možnost uzavření tzv. arbitrážní sázky.

Prohlášení

Prohlašuji, že jsem svou práci vypracoval samostatně a použil jsem pouze prameny a literaturu uvedené v seznamu bibliografických záznamů. Nemám žádné námitky proti zpřístupňování této práce v souladu se zákonem č. 121/2000 Sb. o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon) ve znění pozdějších předpisů.

V Praze dne 20. února 2026

Gabriel Hamrle

Poděkování

Poděkování.

Abstrakt

Abstrakt.

Klíčová slova

extrahování dat, sázková kancelář, arbitrážní sázka, GET požadavek

Abstract

Abstract.

Keywords

data extraction, betting shop, arbitrage bet, GET request

Obsah

1	Teoretická část	3
1.1	Motivace	3
1.1.1	Počítání arbitrážní sázky	3
1.2	Existující řešení	4
1.3	Metody sběru dat z webové stránky	5
1.3.1	HTTP GET metoda	6
1.3.2	HTTP POST metoda	6
2	Implementace	7
2.1	Nalézání vhodných GET metod	7
2.2	Jak získat GET metodu v C#	8
2.3	Struktura ukládání dat	9
2.4	Rozpoznání stejných zápasů	9
2.5	Uživatelské rozhraní	10
3	Technická dokumentace	11
	Závěr	13
	Seznam použité literatury	15
	Seznam obrázků	17
	Seznam tabulek	18

1. Teoretická část

V této části vysvětlím, jaká je motivace srovnávat kurzy napříč sázkovými kanceláři, co to je surebet a jak ho spočítat. Kdo docílil podobného výsledku a k tomu popíšu základní fungování webových stránek a jak z nich extrahovat data.

1.1 Motivace

Sázkové kanceláře vypisují kurzy na tisíce zápasů nezávisle na sobě, čímž vzniká nemalá šance, že některé z kurzů budou nesprávné – tj. jejich hodnota je vyšší než kurz vyplývající ze skutečné pravděpodobnosti. Díky tomu mohou dvě sázkové kanceláře vypsát na dvě opačné události kurzy natolik vysoké, že pokud člověk vsadí na oba výsledky určité množství peněz, vždy skončí v zisku. Tento fenomén se nazývá surebet nebo arbitrážní sázka.

Další využití srovnávání kurzů se nabízí při plnění bonusů, kde je potřeba prosázet určité množství peněz, aby uživatel získal benefit, typicky ve formě freebetu (sázka zdarma), peněžního obnosu nebo freespínů. V těchto případech je žádoucí najít co nejméně ztrátovou sázku, což právě můj program umožňuje.

Vědět o profitujících či méně ztrátových příležitostech je tedy klíčové, pokud chce člověk porazit sázkové kanceláře.

1.1.1 Počítání arbitrážní sázky

V této podsekcí odvodím pár rovnic, které určí, zda-li mohu udělat surebet. Necht' mám události 1 a 2 s kurzy k_1 a k_2 , přičemž jedna z nich se určitě stane. Pokud mám obnos V a chci vsadit na obě události tak, abych vždy vyhrál (respektive prohrál) stejné množství peněz, musí na první vsadit $\frac{V}{k_1}$ a na druhou $\frac{V}{k_2}$, přičemž abych profitovat, tak musí platit, že

$$\begin{aligned} \frac{V}{k_1} + \frac{V}{k_2} &< V \\ \Leftrightarrow \frac{1}{k_1} + \frac{1}{k_2} &< 1. \end{aligned} \tag{1.1}$$

Zobecněním na n kurzů dostaneme, že musí platit:

$$\sum_{i=1}^n \frac{1}{k_i} < 1, \tag{1.2}$$

kde k_i je i -tý kurz na i -tý výsledek a alespoň jeden z výsledků 1 až n vždy nastane. Zhodnocení původního vkladu p lze spočítat následovně:

$$p = \frac{1}{\sum_{i=1}^n \frac{1}{k_i}}. \tag{1.3}$$

Vklad na i -tý kurz V_i lze určit takto:

$$V_i = \frac{V \cdot p}{k_i}. \quad (1.4)$$

1.2 Existující řešení

Hledání surebetů není novinkou a v současnosti existuje několik webových stránek a projektů na platformě GitHub, které tuto službu poskytují. Ovšem nezahrnují všechny české sázkové kanceláře (tj. Tipsport, Chance, Fortuna, Betano, Allwyn, Synottip, Kingsbet, MerkurXtip) nebo jsou alespoň částečně placené. V následující tabulce uvádím šest takových webových stránek a sázkové kanceláře, ze kterých extrahuji data. Také uvádím projekty na GitHubu, které se touto problematikou zabývají. Tyto projekty jsem vyhledával skrze vyhledávač Google, je tedy pravděpodobné, že jsem některé z nich mohl přehlédnout. Taktéž jsem netestoval jejich správnost. Zároveň se mi nepodařilo najít program, který by srovnával data z českých sázkových kanceláří a hledal arbitráže.

Vyhledávače surebetů – webové stránky									
Web	<i>Tipsport</i>	<i>Chance</i>	<i>Fortuna</i>	<i>Betano</i>	<i>Allwyn</i>	<i>Synottip</i>	<i>Kingsbet</i>	<i>MerkurXtip</i>	<i>Placený</i>
Oddspedia	NE	NE	NE	ANO	NE	NE	NE	NE	NE
Rebelbetting	–	–	–	–	–	–	–	–	ANO
BetBurger	ANO	NE	ANO	NE	ANO	NE	NE	NE	ANO
Arbmate	–	–	–	–	–	–	–	–	ANO
Oddsportal	ANO	ANO	ANO	ANO	NE	NE	NE	NE	NE
Surebet	ANO	ANO	ANO	ANO	ANO	ANO	ANO	ANO	NE*

Tabulka 1.1: Vyhledávače surebetů – webové stránky

– znamená, že daná stránka nemá transparentní údaje o tom, které sázkové kanceláře podporuje.

* Profit nad 1 % se zobrazí pouze placeným členům.

Jak lze vidět z tabulky webových stránek nabízejících zmíněnou službu, většina z nich uplatňuje restriktci na zobrazení surebetů pro neplatící uživatele. Můj program samozřejmě nemůže konkurovat profesionálním placeným službám jako surebet.com, představuje však open-source alternativu, ke které lze přidat moduly na extrahování dalších stránek, a zároveň zaručuje bezplatnou dostupnost. Největší nevýhodu svého programu oproti alternativám spatřuji v tom, že není schopen rozpoznat arbitráž pro jiné události než základní výsledek zápasů. Výhodou spuštění vlastního programu na lokálním počítači je naopak fakt, že si mohu sám určit frekvenci extrakce, čímž se snižuje šance, že zobrazená data budou zastaralá.

Vyhledávače surebetů – projekty na GitHubu			
Kancelář	Projekt?	Odkazy	Jazyk
Tipsport	ANO	https://github.com/michalskop/tipsport.cz	Python
Chance	NE	–	–
Fortuna	ANO	https://github.com/michalskop/ifortuna.cz	Python
Betano*	ANO	https://github.com/igormartins0301/projeto-scraping-betano	Python
	ANO	https://github.com/hansalemaos/tutorial_raspagem_de_dados_betano/	Python
Allwyn	NE	–	–
Synottip	NE	–	–
Kingsbet	NE	–	–
MerkurXtip	NE	–	–

Tabulka 1.2: Vyhledávače surebetů – projekty na GitHubu

* Oba projekty jsou v portugalsštině, a extrahují tedy data ze stránky betano.com, nikoliv betano.cz. To by ovšem nemělo vadit, neboť se jedná o tutéž platformu.

Pro svůj projekt jsem si vybral sázkové kanceláře Fortuna a Kingsbet, protože obě disponují API, které jsou snadno pochopitelné (viz následující část), a zároveň se na extrahování dat z Kingsbetu není open-source kód. Tento program jsem psal v programovacím jazyku C#, protože jsem chtěl k extrakci zakomponovat rozhraní webové stránky, která se lépe programuje v ASP.NET frameworku, který existuje pouze pro C#. Projekt by určitě šel udělat i v pythnu, ale na větší projekty se podle mého uvážení nehodí.

1.3 Metody sběru dat z webové stránky

Webové stránky vznikly za účelem vytvoření uživatelsky přívětivé a automatizované komunikace mezi uživatelem a organizací (např. firmou, univerzitou, školou atd.). V současné době se ustálil model, kdy si organizace pořídí nebo pronajme server a webovou adresu. Následně uživatel zadá danou adresu do prohlížeče a obdrží soubory od serveru, které se mu zobrazí jako webová stránka. Typicky obdrží HTML soubor se základní strukturou stránky, CSS soubory, které upravují vzhled HTML souboru, a JavaScriptové soubory, které definují interaktivní prvky stránky. Webové stránky dělíme na dva typy. [3]

Pro jednoduchý přenos neměnných informací stačí takzvaná **statická webová stránka**, která je celá předpřipravená a po odeslání základních souborů již neinteraguje s databází. Tento způsob už v dnešní době není populární, ovšem pro jednoduché stránky s neměnnými daty může postačit.

Na složitější úkoly je potřeba využít tzv. **dynamickou webovou stránku**, která se odlišuje tím, že konzistentně komunikuje se serverem skrze požadavky (requesty), které aktualizují data na webové

stránce, aniž by ji bylo nutné znovu načítat, respektive žádat o další web page request. Server k tomu využívá programovací jazyky typu PHP, ASP.NET či JSP, přičemž o komunikaci v prohlížeči se stará skript v JavaScriptu. [1] Všechny sázkové kanceláře mají dynamické webové stránky.

Většina dynamických webů disponuje API (Application Programming Interface), což je soubor funkcí umožňujících interakci se serverem. [4] Nás bude zajímat situace, kdy klient obdrží data o kurzech, která se často (ale ne vždy) posílají ve formátu JSON (JavaScript Object Notation), a to nejčastěji skrze metody **GET** a **POST**. Jestliže identifikujeme správnou metodu, kterou náš prohlížeč využívá k zasílání kurzů ze serveru do našeho počítače, máme vyhráno, neboť takovou funkci můžeme spustit skrze jakýkoliv program a obdržíme tak přímo čistá data o kurzech.

1.3.1 HTTP GET metoda

Metoda HTTP GET žádá server o data bez dalších přidanych informací. Využívá se hlavně při zobrazení dat, která nejsou senzitivní a nepotřebují vstup (input) od uživatele. [2] Často ji tedy využívají kanceláře k zobrazování aktuálních kurzů.

1.3.2 HTTP POST metoda

Na rozdíl od předchozího případu jsou součástí POST requestu data; jedná se tedy o požadavek se vstupem. [2] Přestože není nezbytný, stránka `sport.synottip.cz` využívá metodu POST k přenosu kurzů. Součástí tohoto requestu je pak např. i token, který podle mých odhadů slouží k zamezení automatizovanému extrahování dat z webu skrze boty.

2. Implementace

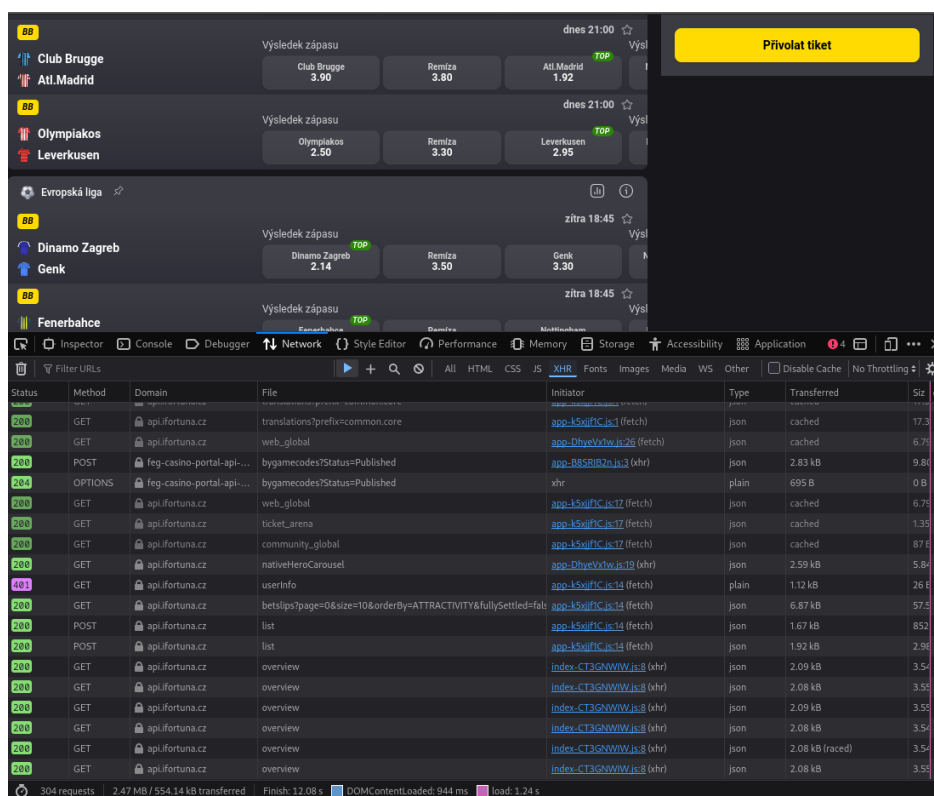
V této části uvedu, jak jsem našel vhodné GET metody, přes které se posílají kurzy. Taktéž vysvětlím jak funguje databáze ukládání zápasů a jak kód rozpoznává stejné zápasy.

2.1 Nalézání vhodných GET metod

Jak již bylo zmíněno v minulé části, většina webových stránek komunikuje se svým serverem skrze API. Tuto komunikaci lze vidět i v prohlížeči za pomoci Inspect tool. Následně popíšu, jak lze tuto komunikaci vidět v prohlížeči Firefox. Ostatní prohlížeče typu Google Chrome budou fungovat velmi podobně.

Po otevření dané stránky, kterou chceme prohledat klikneme kdekoliv na stránce pravým tlačítkem myši a zvolíme možnost Inspect. Následně v horní liště zvolíme Network, čímž uvidíme celou následující komunikaci mezi serverem a stránkou. Tedy předchozí requesty tam nevidíme. Otevření Network lišty lze také docílit za pomoci zkratky Ctrl + Shift + E.

Poté musíme znovu načíst webovou stránku a počkáme, až se načtou všechny potřebné data, což jsou v našem případě kurzy. Následně zastavíme další záznam komunikace skrze tlačítko Pause s ikonou II, aby se nám do toho nepletly např. requesty pro live sázky, které se každou chvílí aktualizují. Nyní vidíme všechnu komunikaci, přes kterou by se mohly kurzy posílat (viz. Obr. 2.1).



Obrázek 2.1: ifortuna.cz s otevřenou network lištou.

Nejjednodušší případ se stává, když se zápasy posílají skrze GET request ve formátu JSON, který

není jakkoliv kódovaný. Jak jsem zjistil, tak sázkové kanceláře Fortuna a Kingsbet využívají přesně tuhle metodu a tedy ukážu pouze co dělat v tomhle případě.

Následně zaklikneme, že chceme vidět pouze XHR - XMLHttpRequest, což je request, kterým se nejčastěji posílají data. (VYSVĚTLIT LÍP) Dále tedy musíme prozkoumat každý z těchto requestů. Někdy mohou být pojmenovány tak, že jde poznat o co jde (např. GetEvents), ale nelze na to spoléhat.

Když člověk klikne pravým tlačítkem myši na daný request a klikne na open in new tab, uvidí JSON formát, který z daného requestu dostanu a taky adresu, přes kterou request udělá. Pokud tímto způsobem rozkliknu několik requestů, které se podle jména zdají relevantní, uvidím několik souborů ve kterých musím nalézt systém jak fungují. Neexistuje jednoznačná metoda jak lze tento systém nazpátek zjistit, ovšem ukážu jak jsem ho zjistil u Fortuny, přičemž u ostatních kanceláří lze pravděpodobně dosáhnout stejného výsledku podobnou metodou, přestože to není zaručené.

Po rozkliknutí několik requestů si doporučuju napsat nějaký základní infomrace o jednom konkrétní zápase (např. název, kurzy) a hledat je v těch dat. Touto metodou jsem zjistil, že data o zápasech jsou v requestu na téhle url:

```
api.afortuna.cz/offer/markets/api/v1_o/fixture/[id_zapasu]/markets/overview
api.afortuna.cz/offer/markets/api/v1_o/fixture/ufo:mtch:1qf-02p/markets/overview
```

Dalším hledáním jsem našel request, kde se nachází ID zápasů:

```
/offer/structure/api/v1_o/tournament/[id_turnaje]/matches?timeFilter=all
/offer/structure/api/v1_o/tournament/ufo:tour:oi-ood/matches?timeFilter=all
```

Pokračováním zjistím, že ID turnamentů se nachází v tomto requestu:

```
/offer/structure/api/v1_o/sport/[id_sportu]/tournaments?timeFilter=all
/offer/structure/api/v1_o/sport/ufo:sprt:oi/tournaments?timeFilter=all
```

A že ID všech možných sportů se nachází v tomto requestu

```
/offer/structure/api/v1_o/sports?timeFilter=all
```

Tedy výsledný kód bude fungovat tak, že si řekne o všechny ID sportů, pak o všechny ID turnajů, poté o všechny ID zápasů a následně tyto zápasy přetransformuje do méj struktury.

2.2 Jak získat GET metodu v C#

Všechny soubory, které extrahují data z externích webů se nachází ve složce ./Background/Match-Finders a jsou spouštěny v souboru OddsFinder.cs

V .NET prostředí lze využít objektu `HttpClient`, který umožňuje získat GET request na základě webové adresy to za pomoci `GetAsync()` metody. Následně jsem z odpovědi vytvořil `JsonNode` objekt. Tato metoda je zde `Arbitra.Background.MatchFinders.KingsbetMatchFinder.GetNodeFromUrl` a zde `Arbitra.Background.MatchFinders.FortunaMatchFinder.GetNodeFromUrl`.

K requestům na webovou stránku Kingsbet bylo potřeba dát header, protože bez něj nedával odpovědi požadavkům.

2.3 Struktura ukládání dat

Všechny soubory, které tvoří strukturu ukládání zápasů, jsou ve složce `./Data Structure`.

V rámci projektu jsem sbíral kurzy pouze na zápasy a to jenom na vítěze zápasu, případně na dvojitip (neprohra týmu nebo absence remízy). Obecná struktura všech událostí je mnohem složitější na uspořádání a tedy není implementovaná v mém projektu. Zde uvádím jednotlivé třídy a jejich využití:

Match - Když se hledají zápasy, tak se ukládají podle této třídy, která obsahuje jméno zápasu, jméno týmu A, jméno týmu B, datum zápasu a tabulku kurzů, která se ukládá jako pole `Odds`. Zároveň obsahuje metodu `Merge()`, která spojuje dva stejné zápasy do jednoho, aby objekt obsahoval informace o všech sázkových kancelářích.

ListOfMatches - list tříd `Match` s funkcí `Merge()`, která spojuje stejné zápasy, které najde v listu. Obsahuje metodu `SplitToEvents()` (stejně jako třída `Match`), která rozděluje zápasy na několik objektů třídy `Event`.

Event - podobná struktura jako `Match`, ale může se stát pouze jeden výsledek z uvedených kurzů. Zároveň lze získat profit ze sázení na tyto události podle vzorců uvedených v motivaci. Taktéž lze tuto třídu serializovat a deserializovat do JSON objektu.

Odd - kurz na konkrétní výsledek vypsaný konkrétní sázkovou kancelář. Obsahuje kurz, sázkovou kancelář, která kurz vypisuje, a odkaz na zápas.

Odds - list tříd `Odd`, který uchovává všechny možné kurzy na jeden konkrétní výsledek zápasu. Obsahuje funkci `Sort()`, která seřadí kurzy od největšího po nejmenší.

MatchOdds - podobná třída jako `Odds`, ale s metodou `Merge()`, která sloučí třídy `Odds`, které reprezentují kurzy na stejné výsledky stejného zápasu.

2.4 Rozpoznání stejných zápasů

Ze zápasů jsem sbíral jména týmů a datum. Protože kanceláře mohou využívat jiný formát názvu týmu, rozhodl jsem se, že každý tým nazvu zkrácenou verzí, kterou vytváří metoda `NormalizeString()` v souboru `/DataStructure/Match.cs`. Zároveň v tom stejném souboru metoda `IsSame()` definuje, kdy jsou oba zápasy stejné. Aby byly, musí platit následující

1. Zkrácený název prvního týmu prvního zápasu je obsažen ve zkráceném názvu prvního týmu

ve druhém zápasu, nebo naopak.

2. Zkrácený název druhého týmu prvního zápasu je obsažen ve zkráceném názvu druhého týmu ve druhém zápasu, nebo naopak.
3. Počet možností, jak zápas může skončit je stejný.
4. Čas, kdy se zápasy konají jsou stejné.

Tento systém rozpoznání stejných zápasů je naprosto arbitrární a nezaručuje stoprocentní úspěšnost. Kvůli identitě času konání je pravděpodobnost na neúspěšnost relativně malá, ovšem pro budoucí vývoj je potřeba vymyslet více robustní metodu.

2.5 Uživatelské rozhraní

Jakožto uživatelské rozhraní jsem si vybral webovou stránku, kterou jsem programoval v ASP.NET frameworku. Jedná se o populární a open source framework na vytváření webových stránek. Uživatel při otevření mého projektu tedy bude obeznámen s rozhraním pro něho známé, tj. rozhraní webových stránek, což je z mého pohledu přívětivější než rozhraní vlastní aplikace, případně výpisu z konzoly. Zároveň zakomponování tohoto projektu do webové stránky umožňuje budoucí uplatnění, pokud se někdy v budoucnu rozhodnu spouštět tento projekt na internet skrze server.

Na hlavní stránce uživatel vidí všechny zápasy s primárními kurzy seřazené od nejvýdělečnějších kurzů, respektive od nejméně prodělečných. Daný kurz si člověk může rozkliknout, čímž se dostane na stránku daného zápasu dané sázkové kanceláře. Také si může rozkliknout zápas jako takový, čímž se dostane do Countru, který umožňuje spočítat, kolik na jaký kurz vsadit, aby byla sázka vždy stejně výnosná.

3. Technická dokumentace

Do projektu jsem zakomponoval dockerfile, který umožňuje spustit můj projekt v docker containeru. K spuštění je potřeba mít nainstalovaný git a docker. Dále stačí naklonovat repozitář do svého počítače a spustit ho v dockeru. Na linuxu toho lze docílit s těmito commandy:

```
git clone https://github.com/Dxlab/Arbitra
```

```
cd Arbitra
```

```
sudo docker compose up --build
```

Po spuštění bude stránka běžet přes port :8080 a tedy stačí navštívit stránku <http://localhost:8080>, (nikoliv https) případně [http://\[ip_adresa\]:8080](http://[ip_adresa]:8080) pro všechny ostatní uživatele na dané síti.

Po spuštění chvíli trvá, než stránka načte kurzy a tedy se tam pár minut žádné nezobrazují. Následně lze vidět zápasy seřazené podle největšího profitu, respektive nejnižší ztráty. Po rozkliknutí daného zápasu lze vidět tzv. Counter, který spočítá na jaký kurz máte vsadit kolik, aby jste vždycky vyhráli, respektive prohráli, stejné množství peněz. Zároveň ukáže celkový profit.

V konzoli se možná vypíší errorry ohledně nedostupného requestu. Ty pouze informují uživatele o faktu, že se kód snažil získat request, který neexistuje. Nejčastěji z důvodu, že se nějaká hodnota chybně označila za např. ID turnaje, zápasu apod. Díky tomu se snažil kód získat request, který neexistuje. Nejedná se ovšem o chybu, která by naznačovala nefunkční systém, spíš ukazuje na neúplně propracovaný kód.

Závěr

Závěr obsahuje shrnutí práce a vyjadřuje se k míře splnění jejího zadání. Dále by se zde mělo objevit sebehodnocení studenta a informace o tom, co nového se naučil a jak vnímal svou práci na projektu.

Seznam použité literatury

- [Gee23] GeeksforGeeks. *What is a Dynamic Website?* 2023. URL: <https://www.geeksforgeeks.org/html/dynamic-websites/>.
- [Gee24a] GeeksforGeeks. *Difference between HTTP GET and POST Methods*. 2024. URL: <https://www.geeksforgeeks.org/php/difference-between-http-get-and-post-methods/>.
- [Gee24b] GeeksforGeeks. *Web Page – A Complete Overview*. 2024. URL: <https://www.geeksforgeeks.org/websites-apps/web-page-a-complete-overview/>.
- [Gee24c] GeeksforGeeks. *What is Web API and Why we use it?* 2024. URL: <https://www.geeksforgeeks.org/blogs/what-is-web-api-and-why-we-use-it/>.

Seznam obrázků

2.1	ifortuna.cz s otevřenou network lištou.	7
-----	---	---

Seznam tabulek

1.1	Vyhledávače surebetů – webové stránky	4
1.2	Vyhledávače surebetů – projekty na GitHubu	5